Informe de validación y pruebas

Despliegue académico de modelo Random Forest | Integración, reproducibilidad y límites de uso

1. Propósito y configuración evaluada

Verificar que la aplicación contenedorizada ejecuta inferencias consistentes con la Actividad 6, comunica correctamente los componentes y rechaza entradas inválidas. La evaluación se realizó sobre un entorno de desarrollo GitHub Codespaces mediante Docker Compose; el despliegue local en la laptop corporativa no se ejecutó.

Elemento	Configuración verificada
Dataset	South German Credit, 1,000 registros; 20 variables de entrada.
Partición	800 registros de desarrollo y 200 de prueba reservada.
Algoritmo	Random Forest, 160 árboles, min_samples_leaf=3, random_state=42.
Umbral	0.28 para clasificar riesgo alto.
Tecnologías	Python 3.11, scikit-learn 1.6.1, FastAPI, Streamlit, Docker Compose.

2. Resultados del modelo reproducido

El artefacto serializado se reconstruyó contra las salidas históricas y reprodujo las probabilidades con diferencia máxima del orden de 9.71×10^{-17}. Al ejecutar la API, las 200 predicciones del conjunto de prueba coincidieron con las referencias dentro de la tolerancia fijada por el redondeo JSON.

Indicador	Resultado	Interpretación
ROC-AUC	0.7588	Discriminación moderada en prueba reservada.
Exactitud	63.0%	126 clasificaciones correctas de 200.
Sensibilidad: riesgo alto	78.3%	47 de 60 casos de riesgo alto detectados.
Especificidad	56.4%	79 de 140 casos no altos identificados.
Precisión: riesgo alto	43.5%	47 aciertos entre 108 alertas de riesgo alto.

2.1. Matriz de confusión — prueba independiente

Real / Predicción	No alto	Alto
No alto (n=140)	TN = 79	FP = 61
Alto (n=60)	FN = 13	TP = 47

El desempeño se interpreta desde la perspectiva académica. El modelo no alcanzó el mínimo de especificidad definido en la Actividad 6 (0.60); la especificidad obtenida fue aproximadamente 0.564. Por tanto, se mantiene el dictamen NO-GO para uso bancario real.

3. Pruebas funcionales y de integración

Caso	Comportamiento observado	Estado
GET /health	HTTP 200; estado operativo y 20 variables.	PASS
GET /schema	20 variables y umbral 0.28.	PASS
POST /predict	Respuesta JSON con probabilidad y clase.	PASS
200 solicitudes históricas	Probabilidades y clases consistentes con referencia.	PASS
Frontend → API	Streamlit mostró resultado y JSON coherentes.	PASS
Docker Compose	Servicios API y frontend: healthy.	PASS

4. Casos extremos y control de entrada

Entrada anómala	Respuesta observada
Variable requerida ausente	HTTP 422.
Variable adicional desconocida	HTTP 422.
Importe negativo	HTTP 422.
Texto donde se espera entero	HTTP 422.
Categoría no incluida en el dominio	HTTP 422 después de corrección v2.

Hallazgo y corrección: la primera versión aceptaba una categoría inexistente. Se incorporó validación de dominio desde form_schema.json en FastAPI y se reforzó el test para fallar automáticamente si el servidor deja de rechazarla. Tras reconstruir y reiniciar el contenedor, las 200 predicciones siguieron coincidiendo con las originales.

Alcance de estas pruebas: los rangos numéricos se configuraron con mínimos y máximos observados del dataset y no constituyen política de originación. No se documentan pruebas de carga, seguridad ofensiva, disponibilidad prolongada ni ejecución en producción.

5. Evidencias técnicas disponibles

Archivo / evidencia	Qué acredita
evidence/pruebas_preliminares.txt	Detección original de categoría fuera de dominio.
evidence/pruebas_validacion_v2.txt	Corrección y repetición de tests de entrada.
evidence/pruebas_compose_final.txt	200 predicciones y pruebas exitosas con Compose.
artifacts/metadata.json	Variables, umbral y matriz de confusión histórica.
docker compose ps (captura)	Dos servicios healthy en Codespaces.
Streamlit (captura)	Formulario, inferencia y respuesta JSON visualizada.
evidence/imagenes/01–06 (PNG)	Capturas de modelo, integración, despliegue, corrección de entradas y repositorio público. Ver INDICE_EVIDENCIAS.txt.

6. Evaluación de riesgos y conclusiones

Se corroboró un prototipo funcional: el modelo es reproducible, la API y la interfaz intercambian información vía HTTP, Docker Compose gestiona ambos componentes y los controles básicos de entrada rechazan los casos adversos previstos. La validación conserva trazabilidad con las Actividades 5 y 6.

La aprobación de las pruebas de software no significa que el modelo sea apto para decisiones reales de crédito. Persisten limitaciones de calidad predictiva, representatividad del dataset, calibración, supervisión humana y gobierno del modelo. Recomendación: conservar el uso exclusivamente académico; cualquier eventual despliegue institucional requeriría una nueva validación y autorización formal.

7. Reflexión personal sobre el ejercicio

Considero que esta fase demostró la diferencia entre software funcional y modelo apto para decisiones bancarias. Ante las restricciones del equipo, prioricé un entorno reproducible y sin pagos, así como verificar resultados antes que añadir herramientas. La corrección del control de entradas confirmó la importancia de probar casos adversos. Mantengo el dictamen NO-GO para cualquier uso crediticio real.

8. Reproducibilidad y referencias

Repositorio: https://github.com/jalfredolarrea-debug/Fase2-Despliegue-Riesgo-Crediticio. Datos: South German Credit (UCI), DOI https://doi.org/10.24432/C5QG88.

El 3 de noviembre de 2026 se clonó el repositorio en una carpeta independiente y se ejecutó Docker Compose: ambos servicios terminaron healthy. Las pruebas reprodujeron 200 predicciones y rechazaron entradas inválidas. Evidencias: evidence/auditoria_reproducibilidad.txt y captura 03. La prueba fue remota en Codespaces, no en otra computadora física.